



TL-CSE: Microarchitecture-Compiler Co-design Space Exploration via Transfer Learning

Zheng Wu
Jinyi Shen
Xuyang Zhao
Changxu Liu
wuz20@fudan.edu.cn
School of Microelectronics, Fudan
University
Shanghai, China

Li Shang
lishang@fudan.edu.cn
School of Computer Science, Fudan
University
Shanghai, China

Fan Yang*
yangfan@fudan.edu.cn
School of Microelectronics, Fudan
University
Shanghai, China

Abstract

The system design of domain-specific processors is a challenging task due to the vast architecture-compiler search space and time-consuming simulation processes. Nowadays, architecture space exploration and compiler optimization are conducted in silo. However, due to the interdependence between microarchitecture and compiler, this segmented approach shrinks the hardware-software design space, leading to sub-optimal global outcomes. To solve problems in co-optimization, this paper introduces TL-CSE, a microarchitecture-compiler co-design space exploration framework. It utilizes a bi-level optimization framework with transfer learning techniques to efficiently explore the hardware-software system design space. Results demonstrate that TL-CSE improves the quality of the Pareto optimal system by 31.5% and speeds up the exploration time by 6.1 times compared to previous co-design frameworks.

Keywords: RISC-V, Microarchitecture Design, GCC Compiler, Transfer Learning, Bi-level Optimization, Bayesian Optimization

ACM Reference Format:

Zheng Wu, Jinyi Shen, Xuyang Zhao, Changxu Liu, Li Shang, and Fan Yang*. 2025. TL-CSE: Microarchitecture-Compiler Co-design Space Exploration via Transfer Learning. In *ASPAC'25: Asia and South Pacific Design Automation Conference, Jan. 20–23, 2025, Tokyo, Japan*. ACM, Tokyo, Japan, 7 pages. <https://doi.org/10.1145/3658617.3697543>

1 INTRODUCTION

Domain-specific processors are specialized hardware tailored for specific applications such as artificial intelligence, multimedia, and network data analysis. In the rapidly evolving technological landscape, the demand for domain-specific processors that provide enhanced performance, reduced power consumption, and increased miniaturization is growing. This demand drives the need for swift iterations in the design process of domain-specific processors.

The design and optimization space for domain-specific processors includes both microarchitecture and compiler elements. Specifically, microarchitecture pertains to the intricate design and structure of a processor, including components such as instruction fetch

and decode units, buffers, caches, and computational units. On the other hand, the compiler is responsible for translating programs into processor instructions and scheduling their execution. Examples of compiler optimization decisions include determining the number of instructions for loop-unrolling and the settings for function inlining. More detailed microarchitecture and compiler options are listed in Table 1a and 1b.

To date, optimizations in microarchitecture and compilers have largely been conducted independently [1, 2, 8, 14, 23, 24] as a means to simplify the design space and render the optimization problem more manageable. However, such separation may limit the full potential of optimization opportunities. Microarchitecture and compilers are, in fact, inherently interdependent [10]. For instance, utilizing a compiler's unroll option can increase instruction parallelism in processors with wider decode units, but it can also cause increased cache misses and stalling in processors with limited registers and cache sizes. Consequently, architectures that could significantly benefit from compiler tuning may be overlooked, resulting in potentially sub-optimal performance outcomes.

This work addresses the microarchitecture-compiler co-design challenge to fully capitalize on optimization opportunities. The primary obstacle is the vast design space coupled with high computational complexity. Specifically, both microarchitecture and compiler present numerous optimization options, and their combined variations can lead to a combination explosion. This issue is compounded by multiple design metrics, such as performance, power, and area. Optimizing such a Pareto front can be an arduous task. Additionally, evaluating the performance of each microarchitecture-compiler optimization candidate involves time-consuming simulations, which further complicates the problem.

To tackle the aforementioned computational complexity challenge, this work presents TL-CSE, a transfer learning-based compiler-microarchitecture co-optimization framework for domain-specific microprocessors. This work makes the following contributions:

- To the best of our knowledge, it is the first framework that defines the microarchitecture-compiler co-optimization challenge as a bi-level optimization problem and proposes a bi-level optimization algorithm to efficiently explore the co-design space.
- We propose a transfer learning approach that leverages the interdependence between microarchitecture and compiler to enhance compiler optimization. We transfer the compiler optimization knowledge of adjacent microarchitectures to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASPAC'25, January 20–23, 2025, Tokyo, Japan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0635-6/25/01

<https://doi.org/10.1145/3658617.3697543>

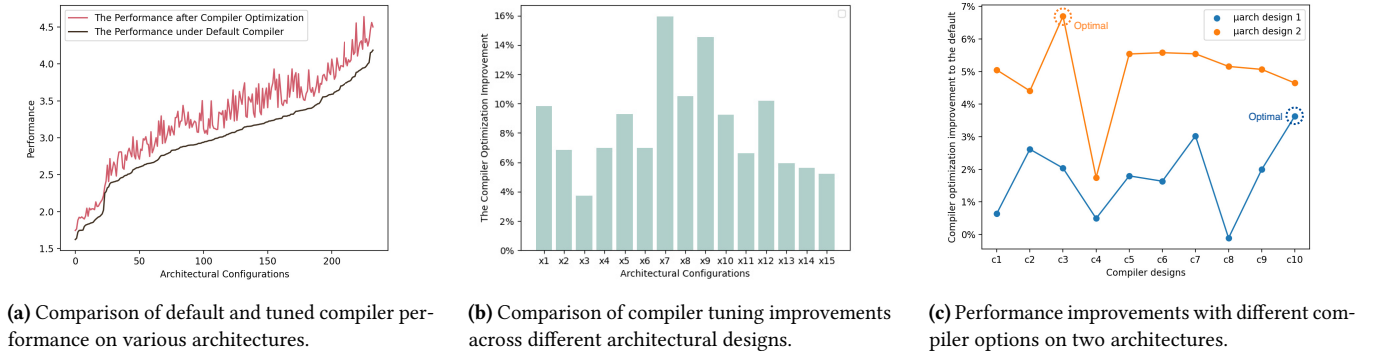


Figure 1. Case study about the correlation between microarchitecture and compiler.

enable efficient initial sampling and robust model prediction, thereby improving the quality of the co-optimization solutions.

- We design an early stopping strategy based on a probabilistic model that predicts the convergence point of compiler optimizations within specific microarchitectures. This allows for early termination of the optimization process, significantly enhancing the efficiency of the co-optimization efforts.
- We conduct experiments on the gem5 processor model with GNU compilers. The results show that our method achieves $6.1\times$ acceleration and 31.5% Pareto front improvement compared to the baseline methods.

The rest of this paper is organized as follows. Section 2 gives an example to explain the interdependence between microarchitecture and compiler. Section 3 introduces the search space of the microarchitecture-compiler co-design problem. Section 4 presents TL-CSE, the proposed automated microarchitecture-compiler co-design space exploration framework. Section 5 demonstrates the experiment results. Section 6 concludes the work.

2 Motivational Example

To highlight the drawbacks of separate optimizations of architecture and compiler, we present a case study of architecture and compiler optimization on the gem5 processor simulator and GNU compiler in Figure 1. Firstly, default compiler option combinations do not necessarily yield optimal performance outcomes. As evidenced in [8] and our experiments depicted in Figure 1a and 1b, compiler optimizations can significantly enhance performance. Neglecting compiler optimizations can lead to missed opportunities for optimal performance. Secondly, the improvement of compiler tuning varies with different microarchitectures. The results in Figure 1a and 1b indicate that the performance improvements from compiler tuning vary across architectures. If architecture and compiler optimizations are conducted separately, some architectures that could benefit greatly from compiler tuning may be overlooked, preventing globally optimal outcomes. Thirdly, the optimal compiler settings for one architecture may not be optimal for another. The results in Figure 1c illustrate that the same compiler settings yield markedly different performance improvements across architectures and the optimal compiler configurations also vary between

designs. Therefore, it is necessary to conduct independent compiler optimization processes for different architectures.

3 Microarchitecture and Compiler Co-design Works and Search Space

Processor microarchitecture is the hardware realization of a processor's instruction set architecture (ISA), detailing how a CPU executes instructions and manages data flow. This design incorporates features such as pipeline, caching, and branch prediction, and organizes functional units, memory hierarchies, and data paths within the CPU. Critical elements like pipeline depth, cache size, and branch prediction algorithms significantly impact the processor's performance, power, and area (PPA). Designers must balance these factors to optimize PPA for specific applications. The exploration of microarchitecture faces challenges due to the vast search space and lengthy simulation times required. Efficient search algorithms [2, 23] and rapid simulation techniques [15, 22, 24] are vital in a swiftly evolving technological landscape. For our study, we selected a RISC-V gem5 processor model and defined a configuration parameter search space, as outlined in Table 1a, to analyze how microarchitectural variations affect PPA.

Compiler optimization is essential in designing domain-specific systems. This process involves selecting compiler options that dictate how programs are translated into hardware instructions and scheduled, impacting execution time and energy consumption. Effective compiler optimizations [1, 8, 14] can enhance instruction parallelism, reduce data and control hazards, and improve hardware utilization. This decreases runtime, thereby optimizing both the underlying architecture and the system's overall efficiency. In our research, we follow the GCC flag tuning space outlined in [19] to study the impact of compiler optimization options on the performance of hardware-software co-optimization. The GCC flag options are listed in Table 1b.

4 The Proposed Framework

4.1 Problem Formulation

The domain-specific processor co-optimization problem includes both microarchitecture and compiler optimizations. In the processor system, microarchitecture is the foundation and has a greater impact on PPA than compilers. Therefore, we define the task as a bi-level optimization (BLO) problem, in which compiler optimizations

Microarchitectures	Design Parameters	Descriptions
Pipeline Width	1:8:1	fetch/decode/issue/commit width
Fetch Buffer	16, 32, 64	fetch buffer size in bytes
Fetch Queue	8: 48: 4	fetch queue size in micro-ops
ROBEntries	32: 256:16	reorder buffer entries
Load/Store Queues	20:48:4	load or store queue entries
Inst Queue Entries	16:80:8	instruction queue entries
IntALU	3:6:1	number of integer ALUs
IntMultDiv	1, 2	number of integer multipliers and dividers
FpALU	1, 2	number of floating-point ALUs
FpMultDiv	1, 2	number of floating-point multipliers and dividers
Int/FpRegs	40:30:8	integer or floating-point physical registers
L1i_size in KB	16, 32, 64	level-1 instruction cache size
L1d_size in KB	16, 32, 64	level-1 data cache size
L2_size in MB	0.5, 1, 2	level-2 cache size
L1i_assoc	2, 4, 8	associative sets of level-1 instruction cache
L1d_assoc	2, 4, 8	associative sets of level-1 data cache
L2_assoc	2, 4, 8	associative sets of level-2 cache

(a) Microarchitecture Design Space of the RISC-V Processor Model.

Flag	Flag	Flag	Flag
-fthread-jumps	-falign-jumps	-fgcse-after-reload	-finline-functions
-fcrossjumping	-falign-loops	-funroll-loops	-funroll-all-loops
-foptimize-sibling-calls	-falign-labels	-falign-functions	-fgcse-las
-fcse-follow-jumps	-ftree-vcpr	-freorder-blocks	-fgcse-sm
-fcse-skip-blocks	-ftree-pre	Param values	Values
-fexpensive-optimisations	-funswitch-loops	large-function-insns	1300:3300:200
-fstrength-reduce	-fschedule-insns	large-function-growth	20, 50, 100, 200, 300, 400, 500
-frerun-cse-after-loop	-fschedule-insns2	large-unit-insns	4000:20000:2000
-frerun-loop-opt	-fno-sched-interblock	inline-unit-growth	10:100:10, 200, 300
-fcaller-saves	-fno-sched-spec	max-unroll-times	2:20:2
-fpeephole2	-fgcse	max-unrolled-insns	50:400:25
-fregmove	-fno-gcse-lm		

(b) GNU Optimization Options and Parameter Values.

Table 1. The Architecture and Compiler Parameters in the Co-design Space.

are nested within the microarchitecture as a constraint. The upper-level optimization (ULO) task is microarchitecture optimization, while the lower-level optimization (LLO) task is compiler optimization. There are two types of variables, namely, the compiler flag options ($x_l \in \mathbb{R}^n$) and microarchitecture parameters ($x_u \in \mathbb{R}^m$). The compiler optimization problem at the lower level can be formulated as:

$$\min_{x_l \in \mathcal{X}_l} \text{Latency}(x_u, x_l), \text{ parameterized by } x_u \quad (1)$$

By denoting the value-function as $\psi(x_u) := \min_{x_l \in \mathcal{X}} f(x_u, x_l)$, we can define the solution-set mapping of Eq. 1 as $\mathcal{S}(x_u) := \{x \in \mathcal{X} \mid f(x_u, x_l) \leq \psi(x_u)\}$. Then the microarchitecture-compiler co-design problem can be formally expressed as

$$\min_{x_u \in \mathcal{X}_u} \text{PPA}(x_u, x_l) = \{\text{Latency}(x_u, x_l), \text{Power}(x_u), \text{Area}(x_u)\} \quad (2)$$

where $\text{PPA} : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}^3$ denotes the objective function of co-optimization. A viable solution to the Bi-Level Optimization (BLO) described in Eq. 2 requires a vector composed of ULO and LLO variables. This solution must fulfill all the constraints outlined in Eq. 2, with the LLO variables optimized for the LLO subproblem specified in Eq. 1, considering the ULO variables as parameters. Our proposed TL-CSE framework addresses this BLO problem and provides a set of Pareto optimal solutions [2].

4.2 Overview

Figure 2 illustrates the TL-CSE framework, a bi-level Bayesian optimization (BO) framework based on transfer learning. The ULO conducts architectural space exploration, while the LLO performs compiler optimization. The ULO is based on the multi-objective BO algorithm. Within the constraints of suggested architectures from ULO, TL-CSE performs compiler optimization enhanced by transfer learning at the LLO. The transfer strategy based on the dynamic distance function and adaptive learning rates is designed to guide transfer learning. The Initial Transfer Sampling and Transfer Ensemble Surrogate Learning algorithms enhance the optimization effectiveness by transfer-learning from the LLO Knowledge base. During LLO, the ML-based early stopping strategy predicts the convergence point and prematurely terminates optimization to enhance efficiency. After the lower-level optimization concludes, TL-CSE updates the optimal performance outcomes in the observed PPA dataset. After predefined iterations, a Pareto-optimal set of system design solutions is obtained.

4.3 Transfer Learning Techniques

4.3.1 Motivation. Within the bi-level optimization framework, we repeatedly optimize compilers under various architectures. Due to the interdependency between microarchitecture and compiler, the compiler optimization improvement exhibits similarities under similar architectures. Figure 3 gives an example of a similar compiler optimization space under a similar microarchitecture design. Theoretically, if we assume that $f(x_u, x_l)$ is continuous or piecewise continuous with respect to x_l , we can identify similarities in the distributions among LLO problems corresponding to neighboring ULO variables x_u . Transfer learning can share useful feature information among similar compiler-tuning tasks and use the information to accelerate compiler optimization for adjacent architectures.

4.3.2 Transfer Strategy. Transfer learning (TL) [3, 5] is a machine learning (ML) approach where knowledge gained from one task is applied to enhance performance on another similar task. Transfer learning involves domains and tasks, where a domain encompasses a feature space $\mathcal{X}$ and a marginal probability distribution $P(\mathcal{X})$, and a task includes labels $\mathcal{Y}$ and a target prediction function $f : \mathcal{X} \rightarrow \mathcal{Y}$. The target function of a task is trained from a sampled dataset $D_n = \{(x_j, y_j)\}_{j=1}^n$. The objective of transfer learning is to utilize knowledge from the source task to aid the learning of the target task. We consider K source tasks, noted as $t_1, t_2, \dots, t_K$, and one target task, noted as t_T . The sampled dataset and prediction functions are $D_{n_1}^{t_1}, \dots, D_{n_K}^{t_K}, D_{n_T}^{t_T}$ and $f^{t_1}, \dots, f^{t_K}, f^{t_T}$.

However, identifying similarities in compiler tuning spaces is essential for ensuring that information transfer through transfer learning is beneficial. Inspired by [5, 7], we propose an algorithm to guide transfer learning, specifying the source task and learning rate to ensure the positive effects of the transfer.

Definition 1. (Dynamic Distance Function) First, we design a dynamic distance function that integrates variables of both ULO and LLO to represent the similarity of the LLO task. [13] and [21] demonstrate the superiority of fractional distance (L_p -norm) in representing distances in high-dimensional spaces and addressing high-dimensional optimization issues. Given the high dimensionality and large parameter values in our optimization problem, we use $p = -\lg(1/m)$ and $p = -\lg(1/n)$ as parameters for the L_p -norm to describe the proximity between the upper and lower levels. The dynamic distance function can be formulated as:

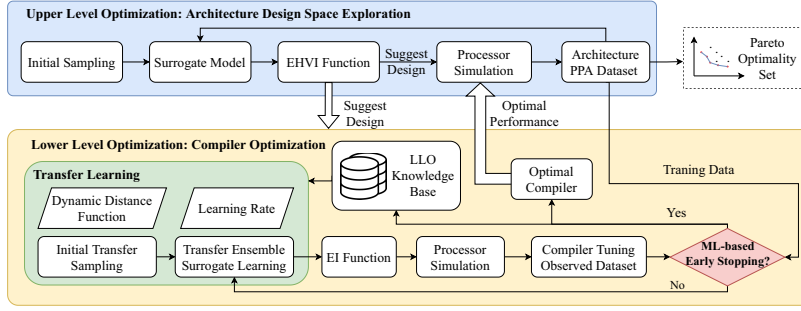


Figure 2. The Overview of TL-CSE Framework.

$$d_{ij}(\text{gen}) = \frac{m}{m+n} L_m(x_u^i, x_u^j) + w(\text{gen}) \frac{n}{m+n} L_n(x_l^{i*}, x_l^{j*}),$$

where $w(\text{gen}) = [1/(1 + \lg(a - \text{gen}/\text{maxgen})) - b]$.

(3)

Here x_u^i and x_u^j denote the i -th and j -th architecture parameters. x_l^{i*} and x_l^{j*} denote the optimal compiler options in the gen -th iteration. maxgen denotes the maximum iteration number of compiler optimization. $a = 1.67$ and $b = 0.66$ are two parameters to limit the range of $w(\text{gen})$.

Definition 2. (Transfer Learning Rate) We design a learning rate function based on the dynamic distance function to describe the contribution of the source task to the target task. Compared to the learning rate function in [5], our function is based on the arctan function. This approach not only ensures the monotonicity of the function but also limits the range of the learning rate. The learning rate can be formulated as:

$$LR_{ij}(\text{gen}) = \left(\frac{2}{\pi} \arctan(-d_{ij}(\text{gen})) + b \right) \times k, \quad (4)$$

where k and b are the limitation parameters to limit the $LR_{ij}(\text{gen})$ to less than 10%. The Dynamic Distance Function and Transfer Learning Rate evaluated the similarity between transfer learning tasks and the contribution of task transfer, forming an important part of the transfer learning strategy.

4.3.3 Transfer Learning-based Compiler Optimization. We apply the transfer strategy to LLO and propose two algorithms based on transfer learning techniques: Transfer Initial Sampling and Ensemble Transfer Surrogate Learning. Transfer Initial Sampling obtains optimal points from neighboring source tasks and uses these points as the initial optimal points for the target task. Ensemble Transfer Surrogate Learning acquires surrogate models from neighboring source tasks. These models are then combined with the surrogate model of the target task using learning rates to create a weighted ensemble, serving as the surrogate model for LLO. The ensemble prediction is formulated as:

$$f_{\text{ensemble}}(X) = \left(1 - \sum_{j=1}^{N'} LR_{JT}(i) \right) * f_T(X) + \sum_{j=1}^{N'} (LR_{JT}(i)) \cdot f_j(X) \quad (5)$$

where f_T denotes the surrogate model prediction of the target LLO task and f_j denotes the j -th source LLO model prediction.

The LLO algorithm with transfer method is presented as Algorithm 1. t_i denotes the i -th task knowledge which includes the

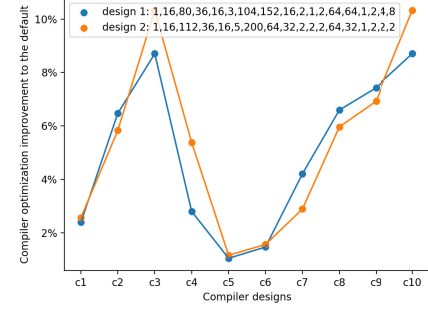


Figure 3. Comparison of compiler opt space across various architectures.

observed dataset $\{X_i, Y_i\}$, surrogate model f_i , and the corresponding upper-level params $x_u^{(i)}$. Lines 1-6 outline the Transfer Initial Sampling algorithm, while lines 7-21 describe the BO algorithm based on Ensemble Transfer Surrogate Learning. Transfer Initial Sampling first calculates the distances between the historical tasks and the target task, as seen in line 1. It selects the closest M' tasks as source tasks and chooses $M' - i$ points from the top i -th closest tasks to include in the initial sampling set. Lines 8-15 detail the Transfer Initial Surrogate Learning algorithm. The top N' source tasks with the highest learning rates are selected for their surrogate models in line 10 and the ensembled prediction can be formulated as Eq. 5. Subsequently, the expected improvement (EI) function for predicted values is computed, followed by identifying the suggested compiler for the next sampling round. After N iterations, the optimal compiler, best performance value, and task knowledge t_T are returned.

4.4 Probabilistic Model-based Early Stopping

Due to the interdependence between microarchitecture and compiler, architectural design could become a bottleneck in tuning compiler performance. Inspired by [19], we notice that microarchitectural information can predict the performance gains from compiler optimization. However, due to limited data in the co-optimization process, the accuracy of the machine learning model may be compromised, affecting the ultimate optimization outcomes. To account for uncertainty in the predictions, we employ a Gaussian Process (GP) model [16] to predict compiler improvement for each microarchitecture design. The formulation of GP is:

$$\begin{cases} \mu(x_*) = \mathbf{K}_*^T \mathbf{K}_n^{-1} \mathbf{y}, \\ \sigma^2(x_*) = \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}_n^{-1} \mathbf{K}_* + \sigma_n^2, \end{cases} \quad (6)$$

where the kernel function $k(x, x') = \sigma_0^2 \exp\left(-\frac{\|x-x'\|^2}{2l^2}\right)$ in GP model. To prevent errors due to model inaccuracies, we introduce the Lower Confidence Bound (LCB) to calculate the convergence point of compiler optimization improvements under specific architectures, as follows:

$$ES = \mu(x_*) - \kappa \cdot \sigma^2(x_*) \quad (7)$$

where κ is a non-negative trade-off parameter to control the balance between exploration and exploitation. As shown in Algorithm 1 line 18, when the minimum value obtained from compiler optimization $y_{\min} < ES$, the LLO will stop and return the results. This approach allows us to terminate the optimization prematurely when the compiler optimization has converged, thereby enhancing the efficiency of the co-optimization process. Experimental results

Algorithm 1 Framework of Transfer Learning-based LLO**Require:**

- The ULO parameters $x_u^{(T)}$;
- The LLO search space $\mathcal{X}$;
- The historical LLO task set $t_S = \{t_1, \dots, t_K\}$;
- The early stopping point ES ;

Ensure:**Transfer Initial Sampling:**

- 1: $\{d_{jT}(0)\}_{j=1}^K \leftarrow$ Dynamic distance between t_S and t_T ;
- 2: Sampled Compiler Set $X \leftarrow \emptyset$;
- 3: **for** $i = 0$; $i < \mathcal{M}'$; $i++$ **do**
- 4: $X \leftarrow$ top $\mathcal{M}' - i$ optimal $x_l^{(i)}$ from the i -th closest tasks;
- 5: **end for**
- 6: $Y \leftarrow$ simulated results of X ;

Ensemble Transfer Surrogate Learning-based Opt:

- 7: **for** $i = 0$; $i < N$; $i++$ **do**
- 8: $\{LR(i)_{jT}\}_{j=1}^K \leftarrow$ The transfer learning rate between t_S and t_T ;
- 9: $y_{ensemble} \leftarrow 0$
- 10: **for each** t_j in t_S with top $\mathcal{N}'$ biggest $LR(i)_{jT}$ **do**
- 11: $y_j \leftarrow$ the prediction of the surrogate model in t_j
- 12: $f_{ensemble}(\mathcal{X}) \leftarrow f_{ensemble}(\mathcal{X}) + f_j(\mathcal{X}) \cdot LR_{jT}(i)$
- 13: **end for**
- 14: Train surrogate model f_T with $\{X, Y\}$;
- 15: $f_{ensemble}(\mathcal{X}) \leftarrow f_{ensemble}(\mathcal{X}) + f_T(\mathcal{X}) \cdot (1 - \sum_{j=1}^{\mathcal{N}'} LR_{jT}(i))$
- 16: Calculate $EI(y_{ensemble})$ and optimize it to get the $x_{suggest}$
- 17: $Y \leftarrow Y \cup$ simulated results of $x_{suggest}$
- 18: **if** $y_{min} < ES$ **then**
- 19: **break**
- 20: **end if**
- 21: **end for**
- 22: **return** $y_{min}, x_{optimal}, t_T \leftarrow \{X, Y, f_T, x_u\}$;

indicate that the Early Stopping method can reduce the number of optimization iterations by 15% without compromising the optimization outcome.

4.5 Bi-level Optimization Framework

In Figure 2, we introduce the TL-CSE framework, a bi-level multi-objective Bayesian optimization framework. Both the upper and lower-level optimization algorithms utilize random forest models [11] as surrogate models. The random forest model excels in robustness, particularly adept at handling discrete problems and diverse data structures. The random forest model can be converted into a probabilistic model, formulated as follows:

$$\begin{cases} \mu(x_*) = \frac{1}{N} \sum_{i=1}^N T_i^*, \\ \sigma^2(x_*) = \frac{1}{N} \sum_{i=1}^N (T_i^* - \mu(x_*))^2, \end{cases} \quad (8)$$

where T_i^* represents the prediction of the i -th tree and N denotes the number of trees. The ULO is a multi-objective optimization problem, requiring trade-offs among PPA metrics. Hypervolume is commonly used to assess the quality of the Pareto front. TL-CSE utilizes the Expected Hypervolume Improvement (EHVI) [18] acquisition function to select recommended designs during iterations. Its formula is:

$$EHVI(x) = \mathbb{E}_{P(f(x))} [HV_{y_{ref}}(\mathcal{P}(y) \cup f(x)) - HV_{y_{ref}}(\mathcal{P}(y))],$$

$$HV_{y_{ref}}(\mathcal{P}(\mathcal{Y})) = \int_{\mathcal{Y}} \mathbb{I}[y \succcurlyeq y_{ref}] [1 - \prod_{y_* \in \mathcal{P}(\mathcal{Y})} \mathbb{I}[y_* \not\prec y]] dy, \quad (9)$$

where $\mathcal{P}(\mathcal{Y})$ is the Pareto-optimal set, $f(x)$ means the prediction of x from ensemble surrogate models and y_{ref} denotes the reference point which is dominated by each element of $\mathcal{P}(\mathcal{Y})$. The expectation $\mathbb{E}_{P(f(x))}$ is estimated using the Monte Carlo method, which samples and averages the hypervolume improvement values.

5 EXPERIMENTS

5.1 Experiment Settings

To assess the effectiveness of our algorithm, we implemented a processor simulation flow using the spec17-gem5-mcpat framework [17]. It includes gem5 [4] with its RISC-V out-of-order (O3) processor model and McPAT (Multi-Core Power, Area, and Timing) [12] for detailed power, area, and timing modeling. We employed the SPEC CPU 2017 benchmark (SPEC17) to evaluate system performance through computationally intensive tasks. We utilize four single-core processor benchmarks: "623.xalancbmk_s," "638.imagick_s," "628.pop2_s," and "600.perlbench_s" to simulate scenarios typical for domain-specific processors. In our experiments, we employ a clock frequency of 1GHz and a process technology of 28nm. Our simulation flow is deployed on a computing cluster powered by Intel(R) Xeon(R) Gold 6230N CPUs, with each design simulation taking approximately 1 hour. Due to limited time and resources, we construct an offline dataset as the baseline method [19], which includes 200 randomly selected microarchitecture designs and 1,000 compiler optimization combinations from the search space, totaling 200,000 data points.

5.2 Baselines and Metrics

We select the latest work on processor microarchitecture-compiler co-optimization [6] as our baseline. It uses an SVM model [6] to predict compiler tuning results but fail to identify optimal configurations, showing an r2 metric of 0.8942 with some error.

We compared separate hardware design space exploration and compiler tuning results. The design space exploration uses the BOOM-Explorer [2] algorithm for 50 rounds, and compiler tuning uses a random optimization algorithm for 300 rounds. The TL-CSE and bi-level BO frameworks are configured with 40 iterations for both inner and outer levels, starting with 10 and 5 initial sampling points, respectively. Experimental results are repeated ten times and averaged to minimize errors. The TL-CMAES [5] are set with 40 rounds for both level and the population is 4. Our method can be accelerated using parallel processing with a batch size of 5 for ULO and LLO, achieving a total of 25 parallels. Other baselines also used 25 parallels to calculate optimization time.

We use the hypervolume (HV) metric and the inverse generative distance (IGD) [20] as evaluation metrics for the quality of the Pareto solution set.

5.3 Results Analysis

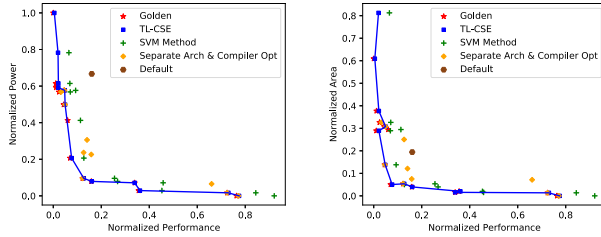
The results in Table 2 demonstrate that the proposed TL-CSE framework, by incorporating the efficient bi-level optimization framework with transfer learning method, acquires the best quality of Pareto optimal solutions among other methods with the shortest time. Compared to the latest processor co-optimization method

Table 2. Comparison of Optimization Results and Time Consumption.

	HV↑	HV Ratio	IGD↓	IGD Ratio	Sampled Points↓	Time(h)↓	Speedup↑
SVM method [6]	1.1245	0%	0.09381	0%	12200	488	1×
Separate Arch [2] & Compiler Opt	1.1362	1.04%	0.09601	-0.023%	3050	122	4×
Vanilla Bi-level Bayesian Opt	1.1544	2.66%	0.07152	23.76%	2250	90	5.42×
TL-CMAES [5]	1.1503	2.29%	0.8309	11.43%	6400	256	1.91×
TL-CSE	1.1745	4.45%	0.06414	31.63%	2250	90	5.42×
TL-CSE w/ Early Stopping	1.1705	4.09%	0.06576	29.90%	1949.39	80	6.1×

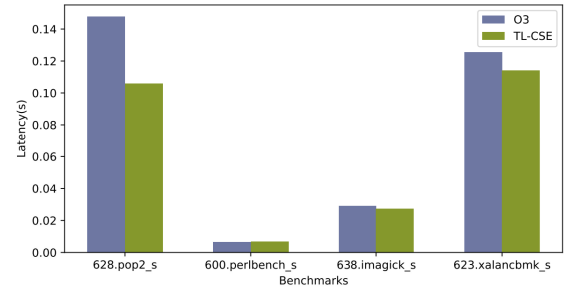
Table 3. Comparison of PPA Results for Microarchitecture Designs Using Various Optimization Algorithms. The Design Params Corresponds to the Params in Table 1a.

	Design Parameters	Latency		Power		Area	
		Value(s)	Ratio	Value(W)	Ratio	Value(mm ²)	Ratio
Default Design & O3	[8, 64, 64, 192, 32, 32, 6, 2, 2, 2, 256, 256, 32, 32, 1, 4, 4, 4]	0.2729	0%	11.6115	0%	12.9454	0%
DSE & Tuning	[5, 64, 20, 224, 40, 80, 5, 1, 1, 1, 120, 240, 32, 16, 1, 2, 4, 4]	0.2659	2.57%	9.1301	21.37%	11.9635	7.58%
SVM Method [6]	[6, 64, 24, 160, 36, 24, 4, 2, 2, 1, 168, 264, 32, 64, 2, 4, 2, 8]	0.2561	6.16%	9.8645	15.05%	14.2892	-10.38%
TL-CSE	[5, 64, 16, 80, 40, 72, 6, 1, 2, 2, 64, 48, 64, 64, 1, 2, 2, 8]	0.2429	10.97%	8.4482	27.24%	11.0148	14.91%

**(a)** The Pareto frontier in performance-power space. **(b)** The Pareto frontier in performance-area space.**Figure 4.** The Pareto frontier comparison among various co-optimization algorithms.

[19], it shortens the time by 5.42 times and improves the quality of the Pareto solution by 31.5%. By incorporating early stopping, the TL-CSE framework reduces sampling iterations by 15% without affecting the overall performance, thereby achieving a 6.1× speedup compared to the baseline. However, the SVM approach [6] is not only time-consuming but also fails to attain high-quality Pareto solutions of hardware-software co-optimization due to the model error. Additionally, the separate microarchitecture and compiler optimization method yielded inferior results, indicating that its solution set is far from the true solution set. This is because it narrows the exploration space and overlooks microarchitecture designs with significant compiler optimization potential. The TL-CMAES [5] is time-consuming because, although it also employs transfer learning for acceleration, the CMA-ES algorithm [9] has lower sampling efficiency than BO.

Figure 4 illustrates the Pareto sets obtained from various optimization methods. The results from TL-CSE closely align with the Golden Pareto Front, in contrast, the Pareto front obtained by the SVM method is significantly distant from the Golden values. The results of separate optimization feature significantly fewer points compared to the Golden Pareto Front and lack representation in the upper left corner of the Pareto Front. This area represents processor designs with extremely low latency on the Pareto front, indicating that the separate optimization method overlooks the microarchitectures with substantial compiler tuning potential. 'Default' represents the combination of the gem5 O3 processor's default microarchitecture and GNU compiler with the '-O3' option. The

**Figure 5.** Optimized benchmark execution latency compared to the default '-O3' compiler.

designs explored by TL-CSE outperform the default system design across PPA metrics.

Table 3 lists the Pareto-optimal microarchitecture-compiler designs identified by various algorithms. Compared to the default O3 processor model provided by gem5, the optimal designs found by TL-CSE exhibit up to a 27.24% improvement in PPA metrics, surpassing the enhancements achieved by other methods. Figure 5 displays a Pareto optimal solution of compiler optimizations using TL-CSE, which achieves a 17% reduction in total program execution time compared to the O3 compiler.

6 Conclusion

This study confirms the efficacy of TL-CSE, our microarchitecture-compiler co-design space exploration framework that integrates bi-level optimization with transfer learning techniques to enhance the design of domain-specific processors. TL-CSE achieves a 31.5% improvement in system quality and a 6.1 times reduction in exploration time compared to traditional methods, effectively addressing the limitations of segmented hardware-software exploration.

Acknowledgments

This research is supported by National Natural Science Foundation of China (NSFC) Research under Project 92373207 and Project 62090025

References

- [1] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. Opentuner: An extensible framework for program autotuning. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 303–316.
- [2] Chen Bai, Qi Sun, Jianwang Zhai, Yuzhe Ma, Bei Yu, and Martin DF Wong. 2021. BOOM-Explorer: RISC-V BOOM microarchitecture design space exploration framework. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [3] Tianyi Bai. 2023. Transfer learning for Bayesian optimization: A survey. *arXiv preprint arXiv:2302.05927* (2023).
- [4] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. 2011. The gem5 simulator. *ACM SIGARCH computer architecture news* 39, 2 (2011), 1–7.
- [5] Lei Chen, Hai-Lin Liu, Kay Chen Tan, and Ke Li. 2021. Transfer learning-based parallel evolutionary algorithm framework for bilevel optimization. *IEEE TEVC* 26, 1 (2021), 115–129.
- [6] Christophe Dubach, Timothy M Jones, and Michael FP O'Boyle. 2008. Exploring and predicting the architecture/optimising compiler co-design space. In *Proceedings of the 2008 international conference on Compilers, architectures and synthesis for embedded systems*. 31–40.
- [7] Matthias Feurer, Jost Springenberg, and Frank Hutter. 2015. Initializing bayesian hyperparameter optimization via meta-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29.
- [8] Grigori Fursin, Yuriy Kashnikov, Abdul Wahid Memon, Zbigniew Chamski, Olivier Temam, Mircea Namolaru, Elad Yom-Tov, Bilha Mendelson, Ayal Zaks, Eric Courtois, et al. 2011. Milepost gcc: Machine learning enabled self-tuning compiler. *International journal of parallel programming* 39 (2011), 296–327.
- [9] Nikolaus Hansen. 2003. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary computation* 11, 1 (2003), 1–18.
- [10] John L Hennessy and David A Patterson. 2011. *Computer architecture: a quantitative approach*. Elsevier.
- [11] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*. Springer, 507–523.
- [12] Sheng Li. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*. 469–480.
- [13] Rachael Morgan and Marcus Gallagher. 2013. Sampling techniques and distance metrics in high dimensional continuous landscape analysis: Limitations and improvements. *IEEE TEVC* 18, 3 (2013), 456–461.
- [14] Zhelong Pan and Rudolf Eigenmann. 2006. Fast and effective orchestration of compiler optimizations for automatic performance tuning. In *International Symposium on Code Generation and Optimization (CGO'06)*. IEEE, 12–pp.
- [15] Erez Perelman, Greg Hamerly, and Brad Calder. 2003. Picking statistically valid and early simulation points. In *2003 12th International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 244–255.
- [16] Carl Edward Rasmussen. 2003. Gaussian processes in machine learning. In *Summer school on machine learning*. Springer, 63–71.
- [17] Alec Roelke and Mircea R Stan. 2017. Risc5: Implementing the RISC-V ISA in gem5. In *First Workshop on Computer Architecture Research with RISC-V (CARRV)*, Vol. 7.
- [18] Amar Shah and Zoubin Ghahramani. 2016. Pareto frontier learning with expensive correlated objectives. In *International conference on machine learning*. PMLR, 1919–1927.
- [19] Alex J Smola and Bernhard Schölkopf. 2004. A tutorial on support vector regression. *Statistics and computing* 14 (2004), 199–222.
- [20] Niranjana Srinivas, Andreas Krause, Sham M Kakade, and Matthias W Seeger. 2012. Information-theoretic regret bounds for gaussian process optimization in the bandit setting. *IEEE transactions on information theory* 58, 5 (2012), 3250–3265.
- [21] Handing Wang, Licheng Jiao, and Xin Yao. 2014. Two_Arch2: An improved two-archive algorithm for many-objective optimization. *IEEE transactions on evolutionary computation* 19, 4 (2014), 524–541.
- [22] Roland E Wunderlich, Thomas F Wenisch, Babak Falsafi, and James C Hoe. 2003. SMARTS: Accelerating microarchitecture simulation via rigorous statistical sampling. In *Proceedings of the 30th annual international symposium on Computer architecture*. 84–97.
- [23] Xiaoling Yi, Jialin Lu, Xiankui Xiong, Dong Xu, Li Shang, and Fan Yang. 2023. Graph Representation Learning for Microarchitecture Design Space Exploration. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [24] Wu Zheng, Yi Xiaoling, Shang Li, and Yang Fan. 2024. SenseDSE: Sensitivity-based Performance Evaluation for Design Space Exploration of Microarchitecture. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.